

Requisitos Funcionales – Proyecto Intermodular

1. Análisis de requisitos y justificación

El alumnado deberá realizar un estudio previo que incluya:

- **Análisis de necesidades:** descripción del problema a resolver, público objetivo y casos de uso principales.
 - **Análisis de la competencia:** comparación con aplicaciones similares (mínimo 2–3), identificando fortalezas, debilidades y oportunidades de mejora.
 - **Justificación tecnológica:** explicación razonada de las tecnologías, frameworks, librerías, patrones y herramientas seleccionadas.
-

2. Calidad del software

El proyecto debe aplicar buenas prácticas de desarrollo:

- **Patrones de diseño:** al menos 2 aplicados de manera explícita (p. ej., MVC, DAO/Repository, Singleton, Factory, Observer...).
 - **Uso de librerías externas:** selección fundamentada y uso eficiente, evitando dependencias innecesarias.
-

3. Autenticación y autorización

El sistema debe incluir un módulo completo de seguridad:

- **Registro, inicio de sesión y recuperación de contraseña,** con gestión segura (hashing, validaciones, tokens).
 - **Sistema de roles:** mínimo *usuario estándar* y *administrador*, con permisos claramente diferenciados.
-

4. Gestión de entidades principales

Toda entidad esencial del proyecto deberá implementarse con las siguientes funciones:

- **CRUD completo:** alta, baja, modificación y listado.

- **Validación de datos:** tanto en el front-end (cliente) como en el back-end (servidor).
 - **Búsqueda avanzada y filtrado dinámico.**
-

5. Panel de usuario

El usuario autenticado dispondrá de:

- **Acciones y vistas personalizadas** según su rol.
 - **Contenido dinámico**, actualizado en tiempo real o bajo demanda.
-

6. Panel de administración

Los administradores contarán con:

- **Gestión global de usuarios**, roles y permisos.
 - **Gestión completa de los datos del sistema** (métricas, control, supervisión).
-

7. API REST documentada

El back-end deberá exponer una API REST que cumpla:

- **Endpoints claramente definidos**, estructurados por recursos y versión estable (v1).
 - **Respuestas en formato JSON**, con códigos de estado HTTP correctos.
 - **Documentación mediante herramientas como Swagger / Redoc / Postman Collection.**
-

8. Persistencia en base de datos

El diseño de datos deberá incluir:

- **Mínimo 6 tablas** con relaciones (1-N, N-N) y claves externas.
 - **Integridad referencial y normalización adecuada.**
-

9. Interfaz responsive

Toda la parte visual debe:

- **Adaptarse correctamente** a móvil, tablet y escritorio.
 - **Usar un diseño flexible** (grid, flexbox, breakpoints, media queries, etc.).
-

10. Diseño accesible

La aplicación debe cumplir criterios de accesibilidad:

- **Criterios conformidad WCAG 2.1**, etiquetas correctas, contraste adecuado, navegación por teclado, textos alternativos, roles ARIA si son necesarios.
-

11. Despliegue y testeo

El proyecto debe presentarse en entorno real:

- **Despliegue desde Git**, con uso de ramas y control de versiones.
 - **Pruebas automáticas** (mínimo pruebas unitarias y/o de integración) en un entorno configurado o pipeline básico (GitHub Actions, por ejemplo).
-

12. Documentación obligatoria

El proyecto debe incluir:

- **Manual de usuario** (funcionamiento general y ejemplos).
 - **Presupuesto técnico** (estimación de horas, coste y recursos).
 - **Memoria del proyecto**, con arquitectura, decisiones técnicas, diagramas, endpoints, capturas y conclusiones.
 - **Cumplimiento de las normas de licencia**
-

13. Respeto propiedad intelectual

El proyecto debe publicarse obligatoriamente con licencia CC-BY-SA

Debe respetarse la normativa sobre propiedad intelectual:

- **Uso correcto de licencias** de software, recursos gráficos, tipografías, etc.
- **Atribución explícita** de todos los recursos externos utilizados (imágenes, iconos, librerías, documentación...).

